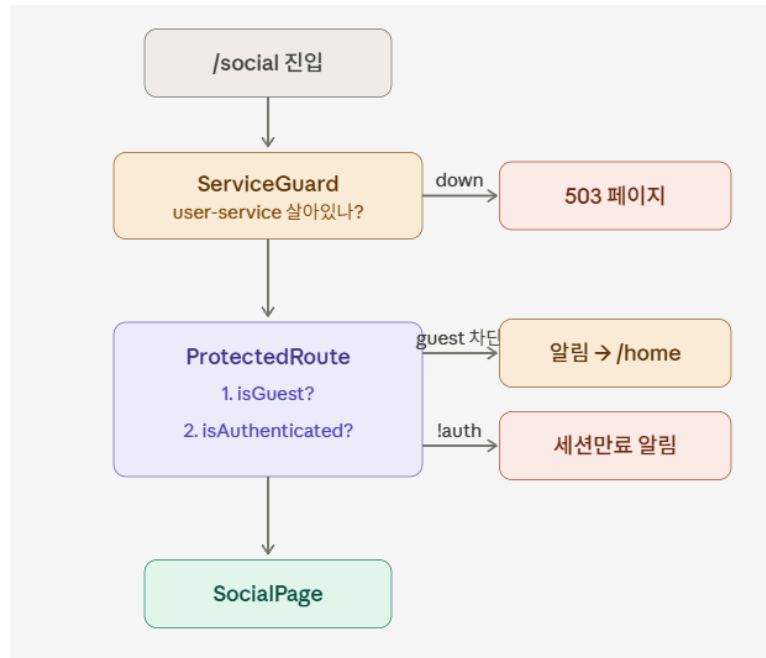


가드 및 게스트

다이어그램



가드 코드 흐름

```
%% 1. 정기 순찰 (Background Polling)
subgraph "배경 작업 (15초마다)"
    Provider[ServiceHealthProvider] -->|"/health/ready"|
요청 | GateW[Gateway]
    GateW -->|상태 확인| AuthS[Auth Service]
    GateW -->|상태 확인| UserS[User Service]
    AuthS & UserS -->|응답| GateW
    GateW -->|JSON 응답| Provider
    Provider -->|메모장 업데이트| ServiceT[(ServiceHealthSt
ore)]
end
```

```

%% 2. 출입 통제 (Route Guard)
subgraph "사용자 페이지 진입"
    User((사용자)) -->|"/social" 클릭| Guard{ServiceGuard}
    Guard -->|메모장 확인| SserviceT
    SserviceT -- "User Service: DOWN" --> Error[ErrorPage:
503 안내]
    SserviceT -- "User Service: UP" --> Auth{ProtectedRoute}

    Auth -- "인증 성공" --> Page[Social Page]
    Auth -- "인증 실패" --> Login[로그인/알림]
end

%% 3. 긴급 차단 (API Error Handling)
subgraph "통신 중 갑작스런 장애"
    Page -->|데이터 요청| Client[apiClient]
    Client -->|실패: 503| ST
    SserviceT -->|즉시 DOWN으로 수정| SserviceT
    SserviceT -->|화면 갱신 이벤트| Guard
end

```

frontend/src/services/serviceHealthStore.ts

에 ServiceHealth전역 변수를 만들

```

export type ServiceHealth = {
  auth: ServiceStatus;
  user: ServiceStatus;
};

let state: ServiceHealth = {
  auth: 'unknown',
  user: 'unknown',
};

```

```
// React 밖의 일반 변수 (상황실 메모장)
let state = { auth: 'unknown', user: 'unknown' };
const listeners = new Set(); // 소식을 듣고 싶어 하는 컴포넌트들의
명단

// 1. 상태를 읽어가는 함수 (getSnapshot)
export const getServiceHealthSnapshot = () => state;

// 2. 명단에 이름을 적고 가는 함수 (subscribe)
export const subscribeServiceHealth = (listener) => {
  listeners.add(listener); // 명단에 추가
  return () => listeners.delete(listener); // 나갈 때 명단에서
삭제
};

// 3. 상태를 바꿀 때 "명단에 있는 모든 사람에게 알려주는" 함수 (emit)
export const setServiceHealth = (next) => {
  state = { ...state, ...next };
  listeners.forEach(fn => fn()); // 종을 울림! "바뀌었으니 다시
읽어가세요!"
};
```

frontend/src/contexts/ServiceHealthContext.tsx에서 상태값을 작성하는 함수

```
const HEALTH_URL = 'http://localhost:8000/health/ready'; //게
이트웨이 health체크만 해도 하위 http 상태 검사 가능
const POLL_INTERVAL_MS = 15_000; // 15초 마다 실행

const parseServiceStatus = (payload: unknown, serviceName: st
ring): ServiceStatus => {
  if (!payload || typeof payload !== 'object') return 'unknow
n';
  const details = (payload as { details?: Record<string, { st
atus?: string }> }).details;
```

```

const entry = details?.[serviceName];
if (!entry) return 'unknown';
if (entry.status === 'up') return 'up';
if (entry.status === 'down') return 'down';
return 'unknown'; //값이 up, down 외에는 모두 unkown으로 반환
};

export const ServiceHealthProvider = ({ children }) => {
  // 여기서 React가 외부 메모장을 지켜보기 시작합니다.
  const health = useSyncExternalStore(
    subscribeServiceHealth, // "나 소식 듣고 싶어!"라고 등록하
    는 법
    getServiceHealthSnapshot // "최신 메모장 내용 좀 보여줘"라고
    읽는 법
  );
};

```

현재 servicehealthstore를 ServiceHealthProvider가 health라는 변수로 상태를 구독(지속 관찰)중임

만약 상태 변화가 발생한다면 provider에

```

{
  "auth": "up",
  "user": "down"
}

```

식의 리턴값으로 보냄.

이 리턴값으로 가드 수행하는 곳은 2군데

UI버튼, serviceGaurd

```

\frontend\src\components\common\Navbar.tsx

const { health } = useServiceHealth();
const userServiceDown = health.user === 'down';

```

```
// 버튼의 비활성화 여부를 결정
<Button disabled={userServiceDown}>소셜 매칭</Button>
```

```
frontend\src\components\common\ServiceGuard.tsx

export default function ServiceGuard({ requires, children })
{
  const { health } = useServiceHealth(); // Context를 통해 health를 가져옴

  // health 정보는 useSyncExternalStore 덕분에 항상 최신입니다.
  if (health[requires] === 'down') {
    return <ErrorPage variant="serviceUnavailable" />;
  }
  return children;
}
```

navbar에서는 만약 유저서비스가 down이라면 버튼을 비활성하여 클릭을 막음.

serviceguard는

```
<ServiceGuard requires="user">
  <ProtectedRoute>
    <SocialPage />
  </ProtectedRoute>
</ServiceGuard>
```

이런식으로 페이지를 감싸서 버튼이 클릭이 되어도 자체적으로 더 검사하여 에러페이지로 리다이렉트 되도록함.

그러나 버튼 클릭 방식이 아닌 url을 직접 수정하여 접속하더라고 apiclient에서 가드를 함.

그러나 apiclient에서는 위에 navbar나 serviceguard처럼 react component가 아닌 따로 isuserservicedown()이라는 함수로 직접 status를 읽는 방식으로 해결

```
// frontend/src/services/apiClient.tsx
import { isUserServiceDown, markServiceDown } from './service
```

```

HealthStore';

const USER_SERVICE_PATH_PREFIX = 'api/users';
const USER_SERVICE_UNAVAILABLE_CODE = 'USER_SERVICE_UNAVAILABLE';

// 요청을 보내려는 주소가 '유저 서비스'를 향하는지 확인하는 함수
const targetsUserService = (url: string) => {
  const trimmed = url.startsWith('/') ? url.slice(1) : url;
  return trimmed.startsWith(USER_SERVICE_PATH_PREFIX);
};

export const apiClient = async (method, url, data) => {

  // --- [1단계: 사전 차단 가드] ---
  // 유저 서비스로 가는 요청인데, 이미 메모장에 'down'이라고 적혀있다면?
  if (targetsUserService(url) && isUserServiceDown()) {
    console.warn(`[apiClient] 유저 서비스가 다운된 상태이므로 요청을 사전에 차단합니다: ${url}`);

    // 서버에 요청을 보내지 않고 즉시 에러 객체를 반환합니다.
    return {
      success: false,
      message: USER_SERVICE_UNAVAILABLE_CODE,
    };
  }

  try {
    // 위 가드를 통과해야만 실제 서버와 통신(axios/fetch)을 시도합니다.
    const response = await axios({ method, url, data });
    return response.data;
  } catch (error) {
    // --- [2단계: 사후 감지 및 메모장 업데이트] ---

```

```

// 만약 요청을 보냈는데 서버가 방금 막 죽어서 503 에러가 왔다면?
if (error.response?.status === 503 &&
    error.response?.data?.message === USER_SERVICE_UNAVAIL
    LABLE_CODE) {

    // 1. 메모장에 'user: down'이라고 적습니다.
    markServiceDown('user');

    // 2. 서버가 죽었으면 serviceHealthProvider가 즉시 재확인 폴
    링을 시작합니다.
    window.dispatchEvent(new CustomEvent('service-health:re
    check'));
}

throw error;
}
};

```

fetchme를 userservice와 authservice로 나눈 이유.

```

<Route
  path="/home"
  element={
    <ProtectedRoute
      isAuthenticated={isAuthReady}
      isGuest={Boolean(user)}
      allowGuest
      onUnauthenticated={handleUnauthenticated}
      revalidateAuth={fetchMe}
    >
      <HomePage />
    </ProtectedRoute>
  }
>

```

```
}
```

현재 app.tsx에서 각 url패스마다 protectedroute로 가드를 만들었습니다.

이때 fetchme로 재인증을하게 되는데(revalidateAuth)

```
const fetchMe = useCallback(async (): Promise<boolean> => {  
  const response = await userService.getMe().catch(() => null); //userService의 getme 사용  
  if (response?.success) {  
    setUser(response.user);  }  
}
```

따라서 유저 서비스다운시 인증도 못함

```
// authService.tsx - me() 신규 추가  
me: async () => {  
  return await apiClient('get', 'api/auth/me');  
  // JWT만 검증, user-service 의존 없음  
},  
  
// useAuthInit.tsx (수정 후)  
const fetchMe = useCallback(async (): Promise<boolean> => {  
  // 1단계: auth-service로 JWT만 검증  
  const authResult = await authService.me().catch(() => null);  
  if (!authResult?.success) {  
    return false; // 진짜 미인증만 여기서 탈락  
  }  
  
  // 2단계: user-service로 프로필 조회  
  const profileResult = await userService.getMe().catch(() => null);  
  if (profileResult?.success) {  
    setUser(profileResult.user); // 정상 케이스  
  }  
}
```



```
    return true;
}
```

fetchme안에 인증 + 프로필 조회를 auth, user서비스 각각 나눠서 구현

게스트 코드 흐름

1. 변경된 DB 테이블

auth_db.auth (auth-service)

- role 컬럼 추가, 게스트는 'guest' 값, 유저는 'member' 값
- loginId / password 가 NULL 허용 (게스트 row 는 둘 다 NULL)

user_db.users (user-service)

- role 컬럼 추가
- nickname unique 제약은 그대로 → 충돌 시 Guest_xxx 재시도로 처리

게스트도 일반 유저와 동일하게 사용. 차이점은 TTL 만 짧음 (30m vs 7d).

사용자가 "게스트로 시작" 버튼을 누르면 `AuthContext` 를 통해 백엔드에 게스트 세션 생성을 요청합니다.

```
// frontend/src/pages/LandingPage.tsx
const handleGuestEnter = async () => {
  // 1. AuthContext의 진입 함수 호출
  const success = await enterGuestMode();

  if (success) {
    // 2. 성공 시 메인 홈으로 이동
    navigate('/home');
  } else {
    // 3. 실패 시 에러 알림 (서버/네트워크 문제 등)
    , , , , ,
  }
}
```

```

    }
  };

  // frontend/src/contexts/AuthContext.tsx
  const enterGuestMode = useCallback(async () => {
    // 백엔드 POST /api/auth/guest 호출
    const result = await authService.guest();

    if (result.success) {
      // 백엔드 응답(닉네임 등)을 상태에 반영
      setUser({
        id: result.user.id,
        nickname: result.user.id,
        isGuest: true,
        ...
      });
      setIsGuest(true);
      return true;
    }
    return false;
  }, []);

```

백엔드 세션 생성 (Auth-Service)

백엔드는 가짜 계정이 아닌, **실제 DB Row**를 생성하여 매칭 시스템 등과의 호환성을 확보합니다.

```

// auth-service/src/auth.controller.ts
@Post('guest')
async guest(@Res({ passthrough: true }) response) {
  const result = await this.authService.guest();

  // 4. [중요] 세션 쿠키 설정 (maxAge 미설정 -> 브라우저 닫으면 삭제)
  const cookieOptions = { httpOnly: true, sameSite: 'lax' };
  response.cookie('accessToken', result.accessToken, cookieOptions);
}

```

```

tions);
    response.cookie('refreshToken', result.refreshToken, cookie
Options);

    return result;
}

```

authservice.guest

```

async guest(): Promise<LoginResult> {
    // 1. Auth 테이블에 비어있는(loginId/PW가 null인) 행 생성
    const newAuth = await this.userRepository.save({ role: 'gue
st' });

    // 2. User-Service에 프로필 생성 요청 (닉네임 중복 시 자동 재시
도)
    const nickname = `Guest_${generateRandomHex()}`;
    await this.userService.init({ id: newAuth.id, nickname, rol
e: 'guest' });

    // 3. JWT 페이로드 구성 (isGuest 클레임 포함)
    const payload = { sub: newAuth.id, id: nickname, isGuest: t
rue };

    return {
        accessToken: this.jwtService.sign(payload, { expiresIn:
'15m' }),
        refreshToken: this.jwtService.sign(payload, { expiresIn:
'30m' }), // 게스트는 짧게 설정
        user: { id: nickname, isGuest: true }
    };
}

```

서비스헬스가 Navbar / ServiceGuard / apiClient 3군데서 가드되는 것처럼, 게스트도 가드.

1. 전역 상태 추출 (Data Source)

모든 가드가 데이터를 가져오는 뿌리입니다.

TypeScript

```
// frontend/src/hooks/useAuth.ts (또는 useContext 직접 사용)
const { isGuest, user, isAuthenticated } = useAuth();
```

2. URL 진입 차단 (ProtectedRoute)

가장 중요한 가드입니다. `/social`, `/myspace` 등 특정 URL로의 접근을 물리적으로 막습니다.

TypeScript

```
// frontend/src/App.tsx 내부 ProtectedRoute 컴포넌트
function ProtectedRoute({ isGuest, allowGuest = false, children, ...props }) {
  const [guestBlockedOpen, setGuestBlockedOpen] = useState(false);
  const navigate = useNavigate();

  useEffect(() => {
    // 1. 게스트인데 이 페이지가 게스트 비허용(allowGuest=false)인 경우
    if (isGuest && !allowGuest) {
      setGuestBlockedOpen(true); // 차단 알림 팝업 ON
    }
  }, [isGuest, allowGuest]);

  if (isGuest && !allowGuest) {
    return (
      <Alert
        open={guestBlockedOpen}
        title={messages.guest.blockedTitle} // ko/en/
        message={messages.guest.blockedBody} // ko/en/
        onClose={() => navigate('/home', { replace: true })}
      />
    );
  }

  return children;
}
```

```
// 홈으로 튕겨냄
    />
  );
}

return children; // 통과
}
```

3. 역방향 진입 차단 (**PublicOnlyRoute**)

이미 게스트로 이용 중인데 `/login` 이나 `/register` 페이지로 가려는 것을 막습니다.

```
// frontend/src/App.tsx 내부 PublicOnlyRoute 컴포넌트
function PublicOnlyRoute({ isAuthenticated, isGuest, children
}) {
  const navigate = useNavigate();

  useEffect(() => {
    // 일반 회원(isAuthenticated)이거나 게스트(isGuest)면 로그인
    페이지 접근 불가
    if (isAuthenticated || isGuest) {
      navigate('/home', { replace: true });
    }
  }, [isAuthenticated, isGuest]);

  if (isAuthenticated || isGuest) return null;
  return children;
}
```

4. 메뉴 UI 차단 (**Navbar**)

버튼을 비활성화해서 사용자가 아예 클릭하지 못하게 만듭니다.

TypeScript

```
// frontend/src/components/common/Navbar.tsx
const menus = [
  { label: '홈', path: '/home', requiresFullAccount: false },
  { label: '소셜', path: '/social', requiresFullAccount: true
}, // 게스트 차단 대상
];

// ... 렌더링 부분
{menus.map((menu) => {
  const isBlocked = menu.requiresFullAccount && isGuest; //
게스트 가드 조건

  return (
    <Button
      key={menu.path}
      disabled={isBlocked} // 클릭 막기
      , , , ,
    >
      {menu.label}
    </Button>
  );
})}
```

게스트 DB 정리 — 두 가지 경로



게스트 데이터는 auth-service 와 user-service 두 DB에 쪼개져 있어서, 어느 경로로 정리하든 양쪽을 모두 지워야 합니다. 사용자가 로그아웃을 누른 경우와 그냥 탭을 닫고 나간 경우를 각각 다른 경로로 처리합니다.

시나리오 A — 로그아웃 클릭 시 (즉시 정리)

1. 프론트엔드: "저 이제 갈게요" (Logout 핸들러)

사용자가 로그아웃 버튼을 누르면 백엔드에 세션 삭제를 요청하고, 프론트 상태를 초기화합니다. 의도된 로그아웃임을 표시해 "세션 만료" 알림과 헷갈리지 않게 합니다.

```
// frontend/src/hooks/Logout.tsx
const handleLogout = async () => {
  try {
    // 1. 백엔드 세션 정리 요청 - 게스트면 백엔드가 즉시 풀 정리까지 수행
    // 합니다.
    await authService.logout();
    // 의도된 로그아웃을 표시 (세션 만료 경고창과 구분하기 위한 플래그)
    sessionStorage.setItem('intent_logout', '1');
  } catch (error) {
    console.error("로그아웃 서버 통신 에러:", error);
  } finally {
    // 2. 프론트엔드 상태 초기화
    setUser(null);
    if (isGuest) exitGuestMode();

    ...

    // 3. 로그인 페이지로 이동
    navigate('/login');
  }
  ...
};
```

2 .auth-service: 토큰의 주인이 누군지 먼저 확인

들어온 refreshToken으로 refresh_sessions 행을 찾아 그 주인의 role을 확인합니다. 게스트면 cron까지 기다리지 않고 즉시 풀 정리, 일반 유저는 기존 흐름 그대로.

```

// auth-service/src/auth.service.ts
async logout(refreshToken?: string) {
  if (!refreshToken) {
    return { success: true, message: 'LOGOUT_NO_REFRESH_TOKEN' };
  }
  const tokenHash = this.hashToken(refreshToken);

  // 토큰의 주인이 게스트면 cron 을 기다리지 않고 즉시 풀 정리.
  // (logout 으로 refresh_sessions row 만 사라지면 cron 의 expires
  // At 검색에 걸리지 않아
  // auth row 와 user-service 프로필이 영구 누락되는 문제 방지)
  const session = await this.refreshSessionRepository.findOne({
    where: { tokenHash } });
  if (session) {
    const owner = await this.userRepository.findOne({ where: {
      id: session.userId } });
    if (owner?.role === 'guest') {
      await this.deleteGuestFully(owner.id);
      await this.addRefreshTokenToBlacklist(refreshToken);
      return { success: true, message: 'LOGOUT_SUCCESS' };
    }
  }

  // 일반 유저 - 기존 흐름 그대로
  await this.refreshSessionRepository.delete({ tokenHash });
  await this.addRefreshTokenToBlacklist(refreshToken);
  return { success: true, message: 'LOGOUT_SUCCESS' };
}

```

3. auth-service: "한 방에 다 지우기" (deleteGuestFully)

user-service 프로필 → auth row 순서로 삭제합니다. user-service가 일시 장애여도 데이터가 누락되지 않도록 cron에 재시도를 위임하는 안전장치를 둡니다.


```

// auth-service/src/auth.service.ts
private async deleteGuestFully(authId: string): Promise<void>
{
  const userServiceUrl =
  this.configService.get<string>('USER_SERVICE_URL') ?? '[http://user-service:4001](http://user-service:4001/)';
  const internalSecret = this.configService.get<string>('INTERNAL_SECRET');

  try {
    // 1. 다른 서비스(user-service) 의 프로필 row 부터 삭제 요청
    await firstValueFrom(
      this.httpService.delete(`${userServiceUrl}/internal/users/${authId}`, {
        headers: internalSecret ? { 'x-internal-secret': internalSecret } : {},
      }),
    );
  } catch (error: any) {
    // user-service 일시 장애 → refresh_sessions.expiresAt 을 과거로 당겨
    // cron 의 검색 조건(expiresAt < now) 에 걸리게 만든다. 다음 10 분 안에 재시도됨.
    await this.refreshSessionRepository.update(
      { userId: authId },
      { expiresAt: new Date(0) }, //만료 날짜를 컴퓨터 date 초기값인 1970년으로
    );
    return;
  }

  // 2. user-service 정리 성공 후, auth row 최종 삭제.
  // onDelete CASCADE 로 refresh_sessions 도 여기서 다 같이 사라집니다.
  await this.userRepository.delete({ id: authId });
}

```

```
}
```

4. user-service: "내부 요청만 받습니다" (보호된 삭제 엔드포인트)

auth-service의 요청을 받아 실제 프로필 row를 지웁니다. 외부인은 접근할 수 없게 Internal Secret으로 보호하고, 일반 유저가 실수로 들어와도 막습니다.

```
// user-service/src/users/user.controller.ts
@Delete('internal/users/:userId')
async deleteGuestUser(
  @Param('userId') userId: string,
  @Headers('x-internal-secret') secret?: string,
) {
  // 1. 내부 서비스끼리만 아는 비밀번호 검증
  const expected = process.env.INTERNAL_SECRET;
  if (expected && secret !== expected) {
    throw new UnauthorizedException('INTERNAL_SECRET_INVALID');
  }
  return this.userService.deleteGuestUser(userId);
}

// user-service/src/users/user.service.ts
async deleteGuestUser(userId: string) {
  // 2. 실제 User Row 삭제 (게스트인지 한 번 더 확인 - 일반 유저 보호
  // 안전망)
  const user = await this.userRepository.findOne({ where: { use
  rId } });
  if (!user) {
    return { success: true, message: 'USER_ALREADY_GONE' }; // 맥
    등 처리
  }
  if (user.role !== 'guest') {
    throw new BadRequestException('NOT_A_GUEST');
  }
  await this.userRepository.delete({ userId });
}
```

```
return { success: true, message: 'GUEST_DELETED' };
}
```

시나리오 B — 탭 닫고 그냥 가버린 경우 (cron 자동 정리)

1. auth-service: "10분마다 순찰하는 환경 미화원" (GuestCleanupService)

사용자가 로그아웃을 안 하고 탭만 닫았을 때, DB가 게스트 데이터로 가득 차는 걸 막아주는 백그라운드 정리 작업입니다. 만료된 refresh 세션을 단서로 게스트만 골라 삭제합니다.

// auth-service/src/guest-cleanup.service.ts

```
@Cron(CronExpression.EVERY_10_MINUTES) // 10분마다 순찰
async cleanupExpiredGuests() {
  const now = new Date();
```

```
  // 1. 만료된(expiresAt < NOW) refresh_sessions 의 user_id 들을 모읍니다.
  const expiredSessions = await this.refreshSessionRepository.find({
    where: { expiresAt: LessThan(now) },
    select: ['id', 'userId'],
  });
  if (expiredSessions.length === 0) return;

  const userIds = Array.from(new Set(expiredSessions.map((s) => s.userId)));

  // 2. 그 중 role='guest' 인 auth row 만 한 번에 추립니다 (일반 유저는 건드리지 않음).
  const guestAuths = await this.authRepository
    .createQueryBuilder('auth')
    .where('auth.id IN (:...ids)', { ids: userIds })
    .andWhere('auth.role = :role', { role: 'guest' })
    .getMany();
  if (guestAuths.length === 0) return;

  const userServiceUrl =
    this.configService.get<string>('USER_SERVICE_URL') ?? '<http://user-service:4001>';
  const internalSecret = this.configService.get<string>('INTERNAL_SECRET');

  for (const auth of guestAuths) {
    try {
      // 3. [중요] 다른 서비스(user-service) 의 프로필부터 삭제 요청
      await firstValueFrom(
        this.httpService.delete(`${userServiceUrl}/internal/users/${auth.id}`, {
          headers: internalSecret ? { 'x-internal-secret': internalSecret } : {},
        }),
      );
    } catch (error: any) {
```

```

    // 만약 user-service 가 죽어있다면? 다음 10분 뒤 cron 에서 재시도. (데이터 정합성 보장)
    this.logger.warn(`user-service 삭제 실패 (${auth.id}), 다음 주기에 재시도`);
    continue;
  }

  // 4. user-service 삭제 성공 후, auth 테이블의 게스트 행 최종 삭제.
  // onDelete CASCADE 로 refresh_sessions 도 여기서 다 같이 사라집니다.
  await this.authRepository.delete({ id: auth.id });
}

}

```

2. user-service: 동일한 엔드포인트로 받음

시나리오 A의 4번과 완전히 같은 핸들러가 처리합니다. 호출 주체가 logout이든 cron이든, user-service 입장에서는 똑같은 "내부 삭제 요청"이라 한 곳에서 받아 처리.

```

// user-service/src/users/user.controller.ts
@Delete('internal/users/:userId')
async deleteGuestUser(
  @Param('userId') userId: string,
  @Headers('x-internal-secret') secret?: string,
) {
  const expected = process.env.INTERNAL_SECRET;
  if (expected && secret !== expected) {
    throw new UnauthorizedException('INTERNAL_SECRET_INVALID');
  }
  return this.userService.deleteGuestUser(userId);
}

```

핵심: 두 경로 모두 동일한 user-service 엔드포인트(DELETE /internal/users/:id)와 동일한 정리 순서(user-service → auth row → CASCADE)를 따릅니다. 차이는 "트리거가 사용자 액션이나 시간이나"뿐. 어느 쪽으로 들어와도 데이터는 일관되게 정리됩니다.